

■ LEVEL 4 • SLIDING WINDOW

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Contiguous subarray/substring + moving range. Expand right; shrink left when invalid.
Reference Examples	Longest Substring Without Repeating Characters • Maximum Sum Subarray of Size K • Minimum Window Substring • Longest Repeating Character Replacement
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 4 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Maximum Sum Subarray of Size K	Fixed Window	#643
2	Longest Substring Without Repeating Characters	Variable Window	#3
3	Minimum Size Subarray Sum	Variable Window	#209
4	Permutation in String	Fixed/Variable Window	#567
5	Find All Anagrams in a String	Fixed Window	#438
6	Longest Repeating Character Replacement	Variable Window	#424
7	Maximum Average Subarray I	Fixed Sliding Window	643
8	Minimum Size Subarray Sum	Variable Sliding Window	209
9	Permutation in String	Frequency Window	567
10	Longest Repeating Character Replacement	Frequency Sliding Window	424

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Maximum Sum Subarray of Size K

Pattern: Fixed Window

LeetCode: #643

■ Input	array of integers and k
■ Output	maximum sum of any contiguous subarray of length k
■ Constraints	$1 \leq k \leq n$
■ Main Idea / Pattern	Keep a running sum. Add right element, remove the element leaving the window.
■■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
long long maxSum(vector<int>& a, int k) {
    long long sum = 0, ans = LLONG_MIN;
    for (int i = 0; i < a.size(); i++) {
        sum += a[i];

        if (i >= k)
            sum -= a[i - k];

        if (i >= k - 1)
            ans = max(ans, sum);
    }
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Longest Substring Without Repeating Characters

Pattern: Variable Window

LeetCode: #3

■ Input	string s
■ Output	length of longest substring with all unique characters
■ Constraints	$0 \leq s.length \leq 5 * 10^4$
■ Main Idea / Pattern	Expand right. If a character repeats, move left until the window is valid.
■■ Time Complexity	$O(n)$
■ Space Complexity	$O(\min(n, \text{charset}))$

■ Optimal C++ Code

```
int lengthOfLongestSubstring(string s) {  
    unordered_set<char> seen;  
    int left = 0, ans = 0;  
    for (int right = 0; right < s.size(); right++) {  
        while (seen.count(s[right])) {  
            seen.erase(s[left]);  
            left++;  
        }  
        seen.insert(s[right]);  
        ans = max(ans, right - left + 1);  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Minimum Size Subarray Sum

Pattern: Variable Window

LeetCode: #209

■ Input	positive integers and target
■ Output	minimum length of a contiguous subarray with sum $\geq$ target
■ Constraints	all numbers are positive
■ Main Idea / Pattern	Expand right; while the sum is enough, shrink from the left and record the best length.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int minSubArrayLen(int target, vector<int>& nums) {
    int left = 0, ans = INT_MAX;
    long long sum = 0;

    for (int right = 0; right < nums.size(); right++) {
        sum += nums[right];

        while (sum >= target) {
            ans = min(ans, right - left + 1);
            sum -= nums[left++];
        }
    }

    return ans == INT_MAX ? 0 : ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Permutation in String

Pattern: Fixed/Variable Window

LeetCode: #567

■ Input	strings s1 and s2
■ Output	whether s2 contains a permutation of s1
■ Constraints	lowercase English letters
■ Main Idea / Pattern	Maintain frequency counts for a window of size s1.length(). Compare the two 26-count arrays.
■ Time Complexity	O(n)
■ Space Complexity	O(1)

■ Optimal C++ Code

```
bool checkInclusion(string s1, string s2) {
    if (s1.size() > s2.size()) return false;
    vector<int> need(26), win(26);
    for (char c : s1) need[c - 'a']++;
    int k = s1.size();
    for (int i = 0; i < s2.size(); i++) {
        win[s2[i] - 'a']++;
        if (i >= k)
            win[s2[i - k] - 'a']--;
        if (i >= k - 1 && win == need)
            return true;
    }
    return false;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Find All Anagrams in a String

Pattern: Fixed Window

LeetCode: #438

■ Input	strings s and p
■ Output	starting indices of anagrams of p in s
■ Constraints	lowercase English letters
■ Main Idea / Pattern	Use a fixed-size frequency window and compare it with p's frequency.
■■ Time Complexity	O(n)
■ Space Complexity	O(1)

■ Optimal C++ Code

```
vector<int> findAnagrams(string s, string p) {  
    vector<int> need(26), win(26), ans;  
    if (p.size() > s.size()) return ans;  
    for (char c : p) need[c - 'a']++;  
    int k = p.size();  
    for (int i = 0; i < s.size(); i++) {  
        win[s[i] - 'a']++;  
        if (i >= k)  
            win[s[i - k] - 'a']--;  
        if (i >= k - 1 && win == need)  
            ans.push_back(i - k + 1);  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Longest Repeating Character Replacement

Pattern: Variable Window

LeetCode: #424

■ Input	uppercase string and k
■ Output	longest substring after at most k replacements
■ Constraints	$1 \leq s.length \leq 10^5$
■ Main Idea / Pattern	Window is valid when window size - most frequent character count $\leq k$.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int characterReplacement(string s, int k) {  
    vector<int> freq(26);  
    int left = 0, maxFreq = 0, ans = 0;  
    for (int right = 0; right < s.size(); right++) {  
        maxFreq = max(maxFreq, ++freq[s[right] - 'A']);  
        while (right - left + 1 - maxFreq > k)  
            freq[s[left++] - 'A']--;  
        ans = max(ans, right - left + 1);  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Maximum Average Subarray I

Pattern: Fixed Sliding Window

LeetCode: 643

Input	nums, k
Output	maximum average of any length-k subarray
Constraints	$1 \leq k \leq \text{nums.size()} \leq 1e5$
Main Idea / Pattern	Keep a running window sum; add right and remove the element leaving the window.
Time Complexity	$O(n)$
Space Complexity	$O(1)$

Optimal C++ Code

```
class Solution {
public:
    double findMaxAverage(vector<int>& nums, int k) {
        int sum = 0;
        for (int i = 0; i < k; i++) {
            sum += nums[i];
        }
        int best = sum;
        for (int i = k; i < nums.size(); i++) {
            sum += nums[i];
            sum -= nums[i - k];
            best = max(best, sum);
        }
        return (double)best / k;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Minimum Size Subarray Sum

Pattern: Variable Sliding Window

LeetCode: 209

Input	target, nums
Output	minimum length of a contiguous subarray with sum $\geq$ target
Constraints	$1 \leq \text{nums.size()} \leq 1e5$; $\text{nums}[i] > 0$
Main Idea / Pattern	Expand right until valid, then shrink left while the sum remains valid.
Time Complexity	$O(n)$
Space Complexity	$O(1)$

Optimal C++ Code

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int left = 0;
        int sum = 0;
        int ans = INT_MAX;

        for (int right = 0; right < nums.size(); right++) {
            sum += nums[right];

            while (sum >= target) {
                ans = min(ans, right - left + 1);
                sum -= nums[left++];
            }
        }

        return ans == INT_MAX ? 0 : ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Permutation in String

Pattern: Frequency Window

LeetCode: 567

Input	s1, s2
Output	whether s2 contains a permutation of s1
Constraints	1 <= s1.size(), s2.size() <= 1e4
Main Idea / Pattern	Maintain character frequencies for a fixed-size window and compare it with s1.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    bool checkInclusion(string s1, string s2) {
        if (s1.size() > s2.size()) return false;
        vector<int> need(26, 0), window(26, 0);
        for (char c : s1) need[c - 'a']++;
        for (int i = 0; i < s2.size(); i++) {
            window[s2[i] - 'a']++;
            if (i >= s1.size()) {
                window[s2[i - s1.size()] - 'a']--;
            }
            if (window == need) return true;
        }
        return false;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Longest Repeating Character Replacement

Pattern: Frequency Sliding Window

LeetCode: 424

Input	s, k
Output	length of the longest substring after at most k replacements
Constraints	1 <= s.size() <= 1e5; uppercase English letters
Main Idea / Pattern	A window is valid when window length - most frequent character count <= k.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int characterReplacement(string s, int k) {
        vector<int> freq(26, 0);
        int left = 0;
        int maxFreq = 0;
        int ans = 0;

        for (int right = 0; right < s.size(); right++) {
            maxFreq = max(maxFreq, ++freq[s[right] - 'A']);

            while (right - left + 1 - maxFreq > k) {
                freq[s[left++] - 'A']--;
            }

            ans = max(ans, right - left + 1);
        }

        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.